

Relatório Técnico: Desenvolvimento do Jogo Batalha Naval em Java, utilizando JavaFX, Scene Builder e os conceitos de Programação Orientada a Objetos (POO)

Thiago Vitor Moreira Maia

Instituto Metr pole Digital – Universidade Federal do Rio Grande do Norte (UFRN) –
Natal – RN – Brasil

thiago.moreira0711@gmail.com

1. Introdu  o

O jogo Batalha Naval   um cl ssico jogo de estrat gia de tabuleiro que envolve dois jogadores tentando afundar os navios do advers rio. Este relat rio t cnico documenta o desenvolvimento de uma vers o digital do jogo utilizando princ pios de Program  o Orientada a Objetos (POO), a linguagem de programa  o Java e a plataforma JavaFX.

2. Abordagem de solu  o do problema

2.1. Projeto Orientado a Objetos

Para desenvolver o jogo Batalha Naval, foi criado um projeto utilizando os princ pios da Program  o Orientada a Objetos (POO). O projeto foi estruturado em v rias classes, cada uma respons vel por uma parte espec fica da l gica do jogo. Abaixo, uma rela  o das classes desenvolvidas e seus respectivos pacotes:

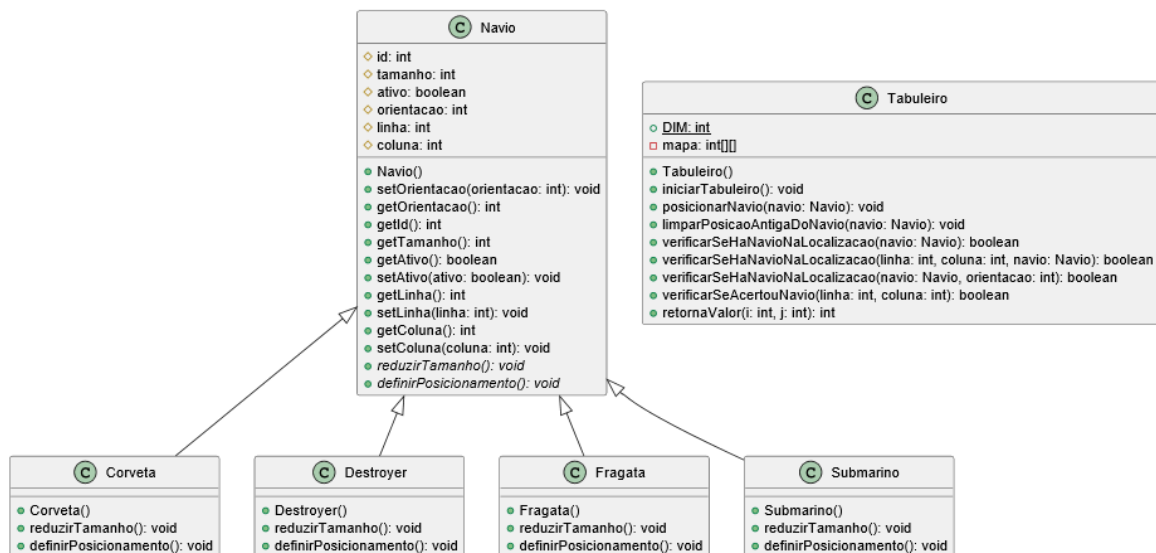
PACKAGE	CLASSE
br.ufrn.imd	MainApp.java
br.ufrn.imd.visao	TelaInicial.fxml
	TelaJogo.fxml
	TelaComoJogar.fxml
	TelaSobre.fxml
br.ufrn.imd.modelo	Tabuleiro.java
	Navio.java
	Corveta.java
	Submarino.java
	Fragata.java
	Destroyer.java
br.ufrn.imd.dao	JogadasDAO.java
br.ufrn.imd.controle	TelaJogoController.java
	TelaInicialController.java

2.2. Decisões de Projeto e Padrões Aplicados

Durante o desenvolvimento do projeto, foram tomadas as seguintes decisões:

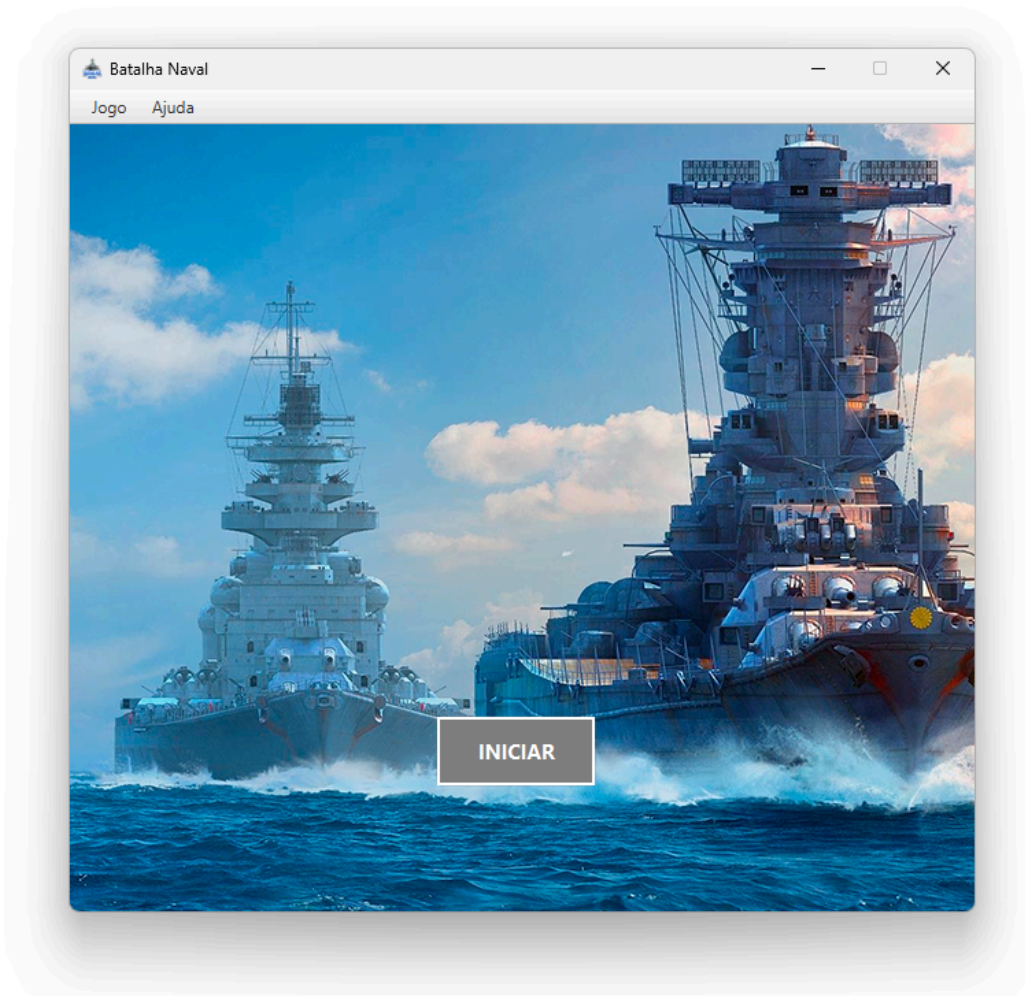
- **Separação de Responsabilidades:** cada classe possui uma responsabilidade clara e distinta, facilitando a manutenção e expansão do código;
- **MVC (Model-View-Controller):** O padrão MVC foi aplicado para separar a lógica de apresentação (View), a lógica de controle (Controller) e a lógica de negócios (Model).

2.3. Diagrama de Classes



O diagrama de classes acima representa a modelagem das classes modelos do jogo Batalha Naval. As subclasses Corveta, Destroyer, Fragata e Submarino herdam a classe abstrata Navio e representam os navios do jogo, com seus respectivos atributos e métodos. A classe Tabuleiro, por sua vez, representa o tabuleiro do jogo e possui métodos necessários para posicionar navios, limpar o posicionamento de navios, verificar se há algum navio em um intervalo da matriz e retornar os índices de uma determinada posição do tabuleiro.

2.4.1. Tela Inicial



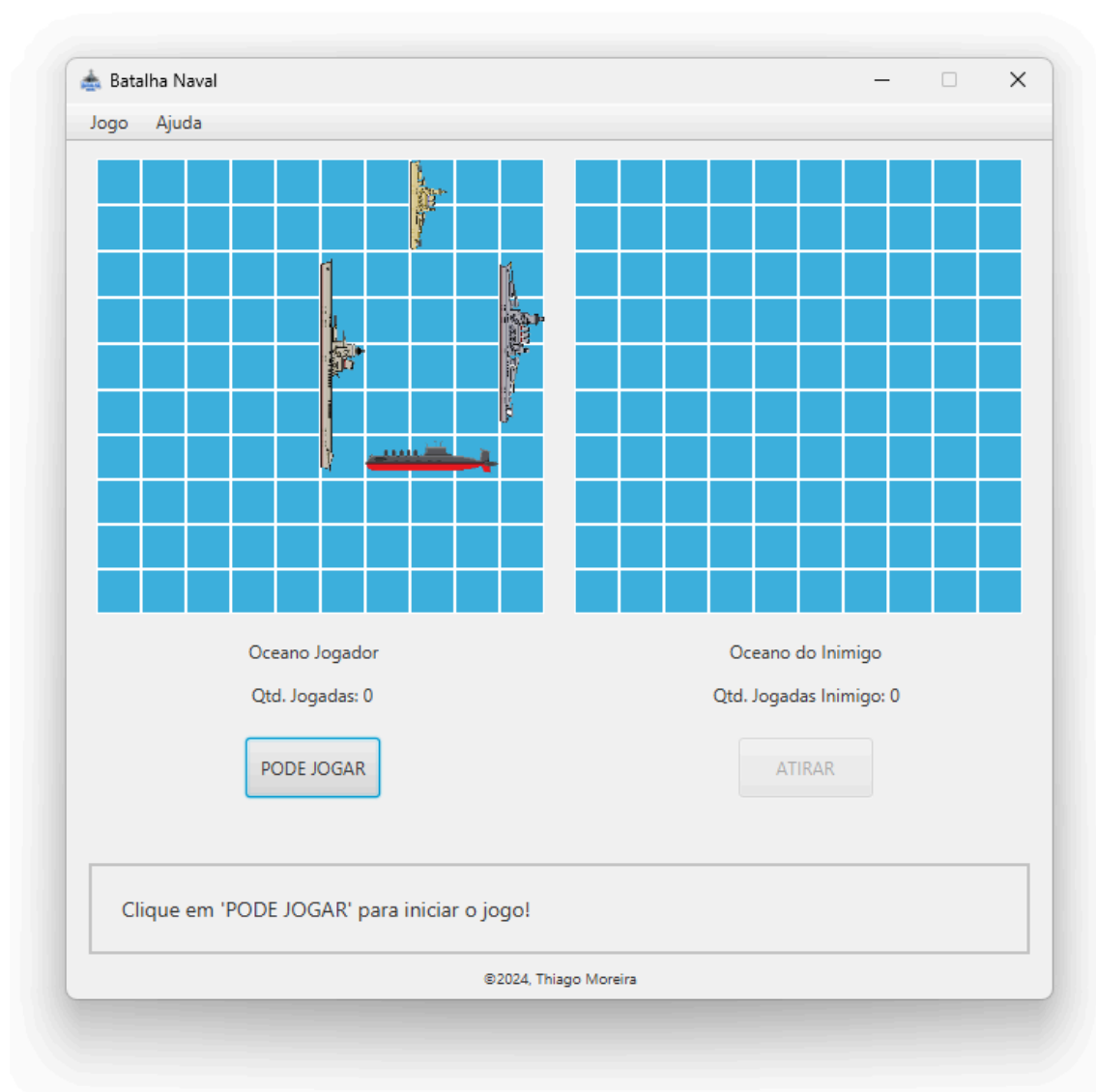
Ao iniciar o programa, a tela inicial é exibida para o jogador. Essa tela foi desenvolvida para servir como uma breve apresentação do jogo e possui os seguintes objetivos:

- estabelecer uma identidade visual para o projeto; e
- gerar uma ambientação para o jogo, contribuindo para a imersão do jogador desde o momento em que ele abre o programa.

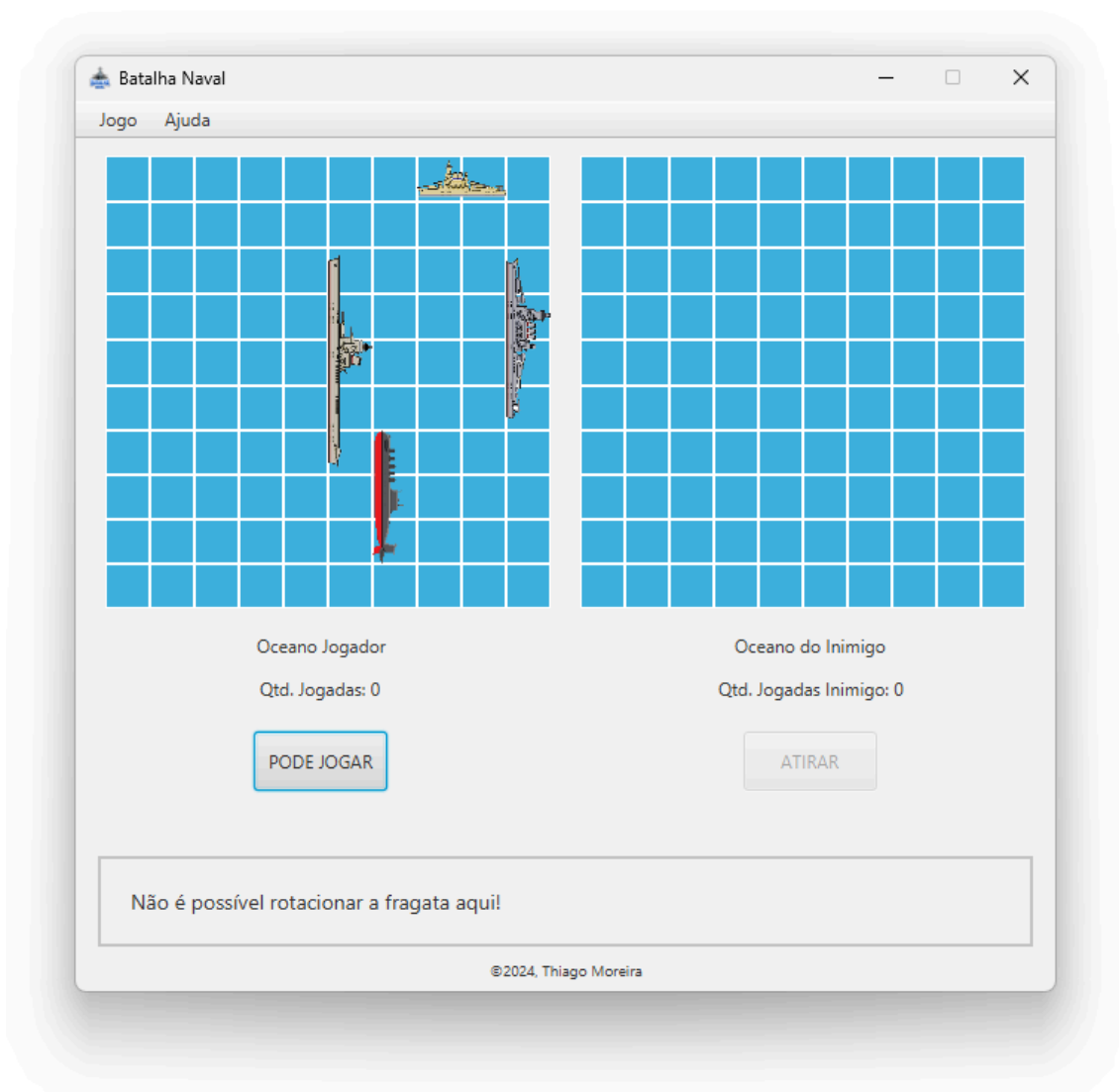
A tela inicial é formada por um **AnchorPane**, ao qual foi adicionado a propriedade de estilo `background-image`, gerando um wallpaper para a tela. A estrutura geral da tela inicial construída no SceneBuilder é a seguinte:

- **AnchorPane**
 - **MenuBar**
 - **Item Jogo**
 - Subitem Novo Jogo
 - Subitem Sair
 - **Item Ajuda**
 - Subitem Como jogar
 - Subitem Sobre
 - **ImageView** (com a logo desenvolvida exclusivamente para o projeto)
 - **Button Iniciar** (ao ser clicado, permite o carregamento da tela do jogo)

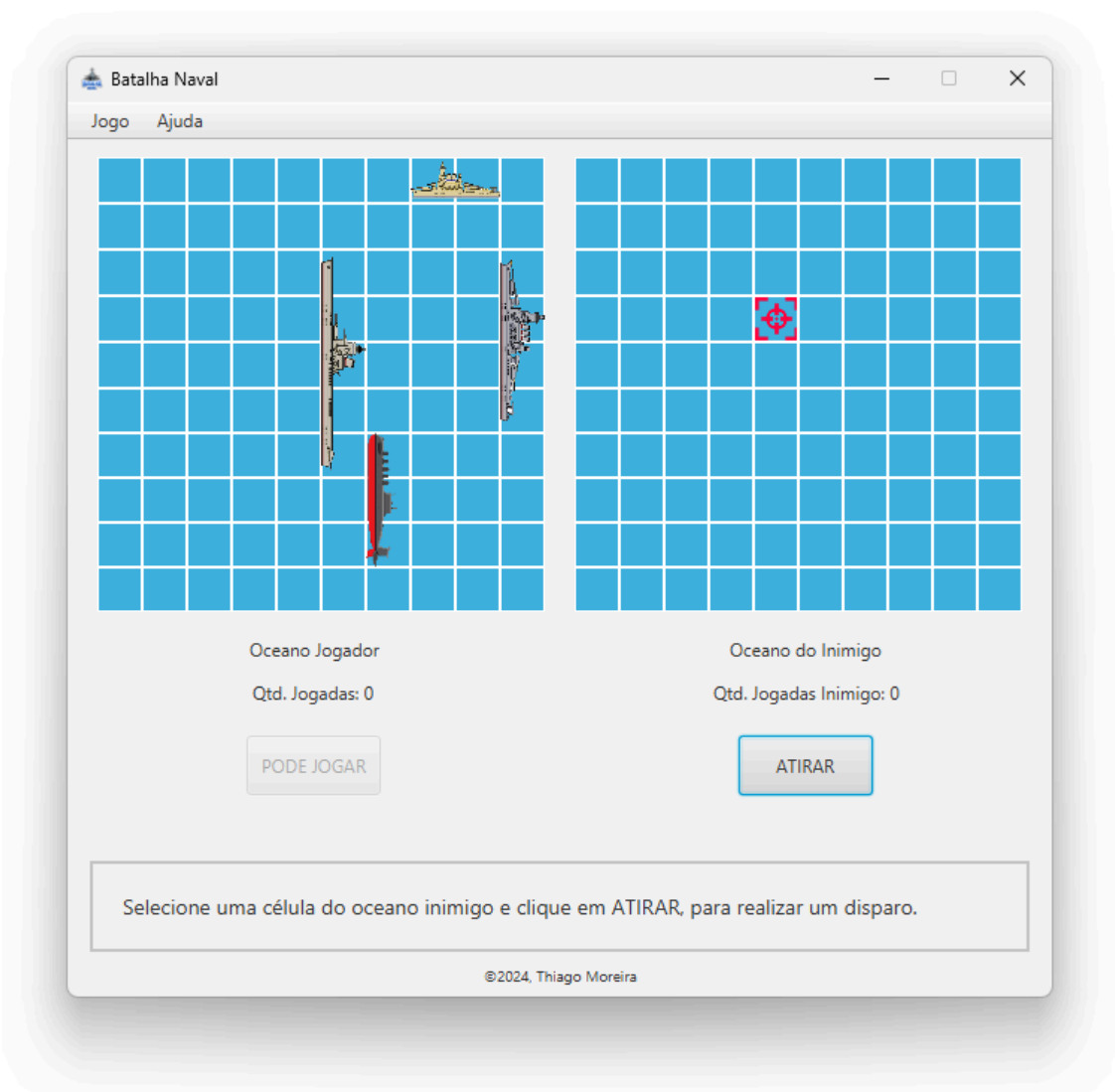
2.4.2. Telas do Jogo



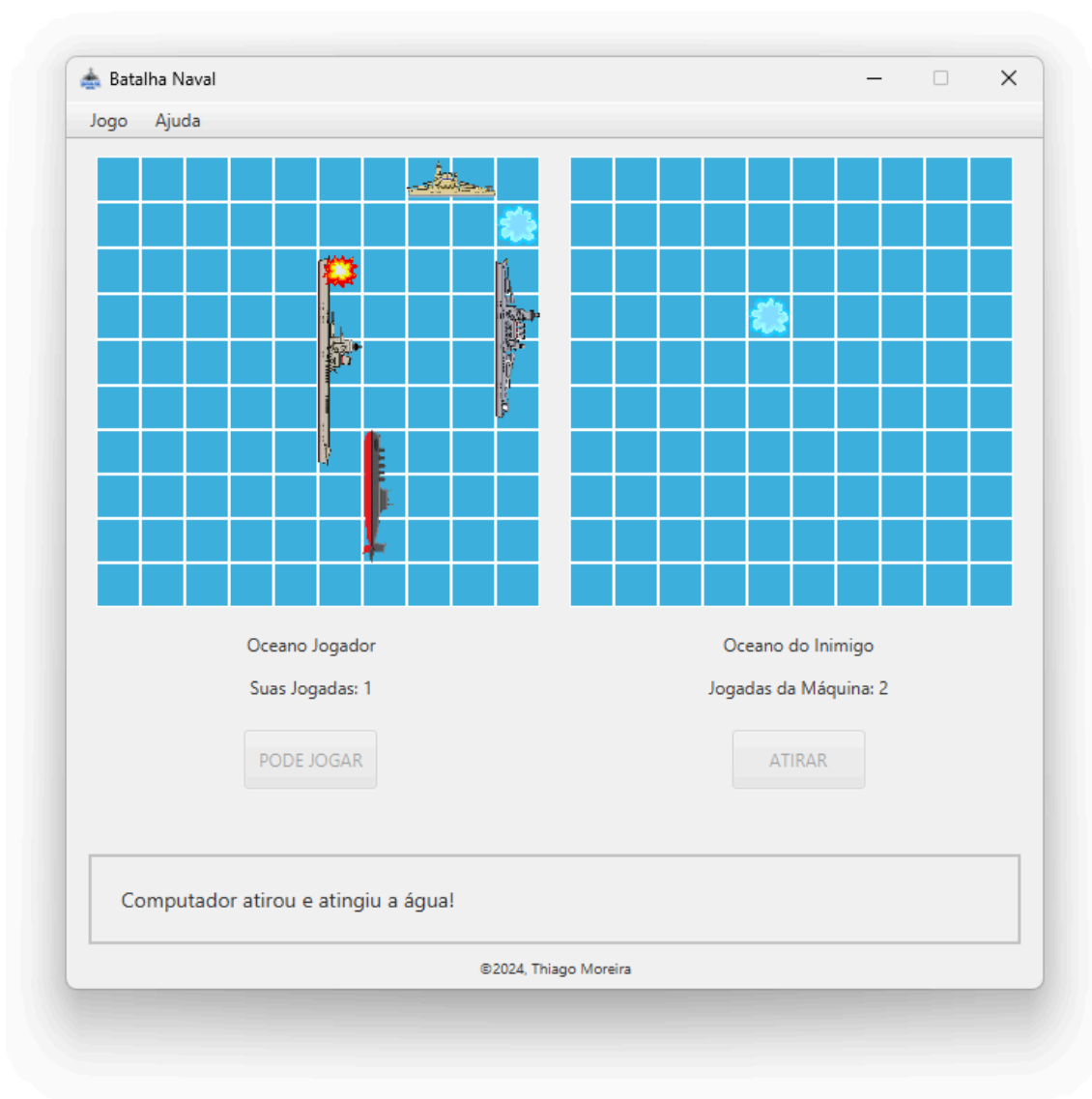
A imagem acima é uma representação da tela do jogo para o usuário. A tela exibe um MenuBar, com as opções Jogo e Ajuda. Abaixo, são exibidos o tabuleiro do jogador e o tabuleiro do inimigo, com as suas respectivas indicações imediatamente abaixo (Oceano Jogador e Oceano do Inimigo). Possui, ainda, indicadores da quantidade de jogadas realizadas pelo jogador e pela máquina, os botões “PODE JOGAR” e “ATIRAR” e um Pannel, em que são exibidas as informações relativas à partida.



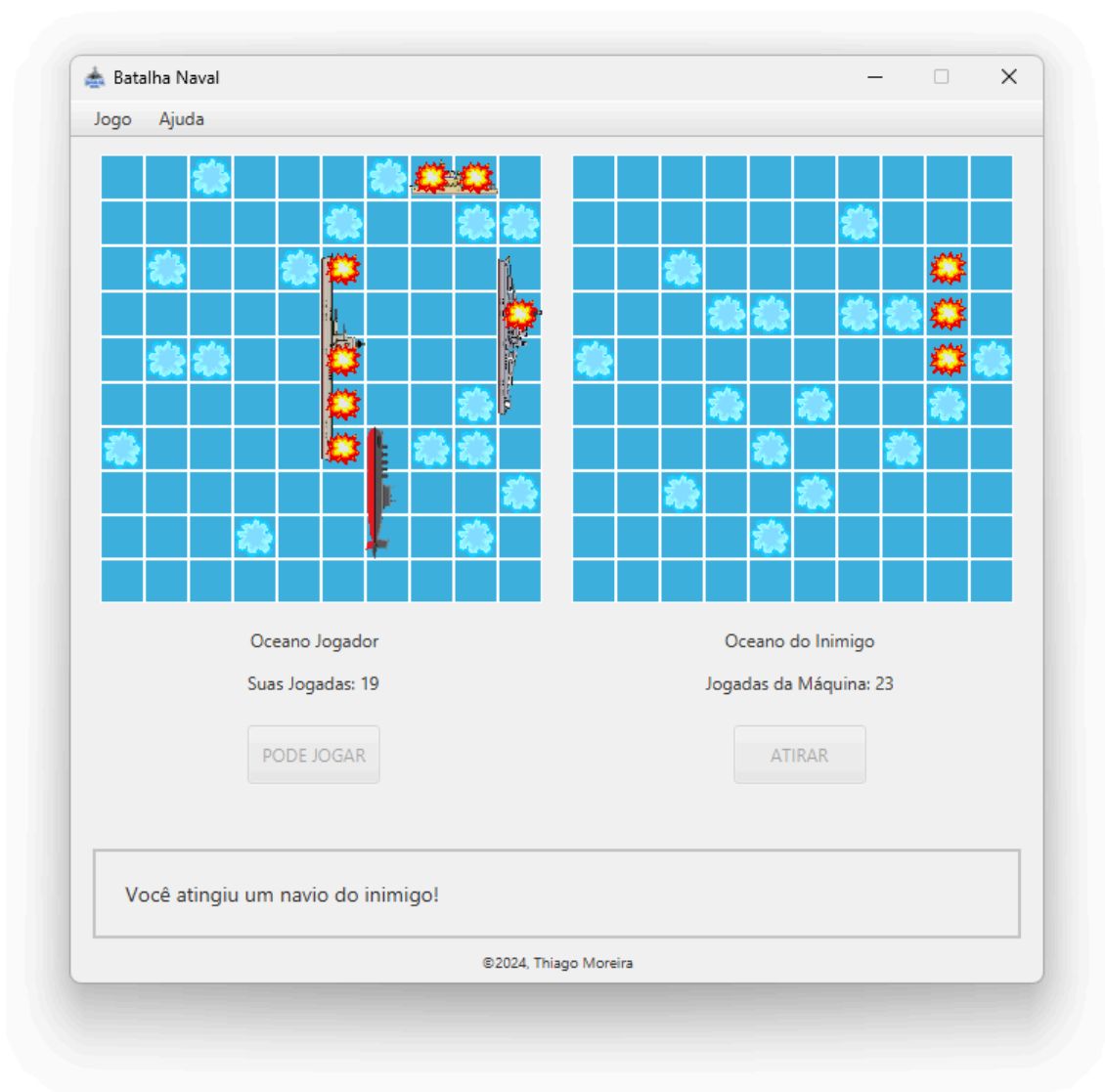
Na imagem acima, a Corveta e o Submarino foram rotacionados. Ao tentar rotacionar a Fragata, é exibida uma mensagem no painel indicando que não é possível realizar o processo de rotação.



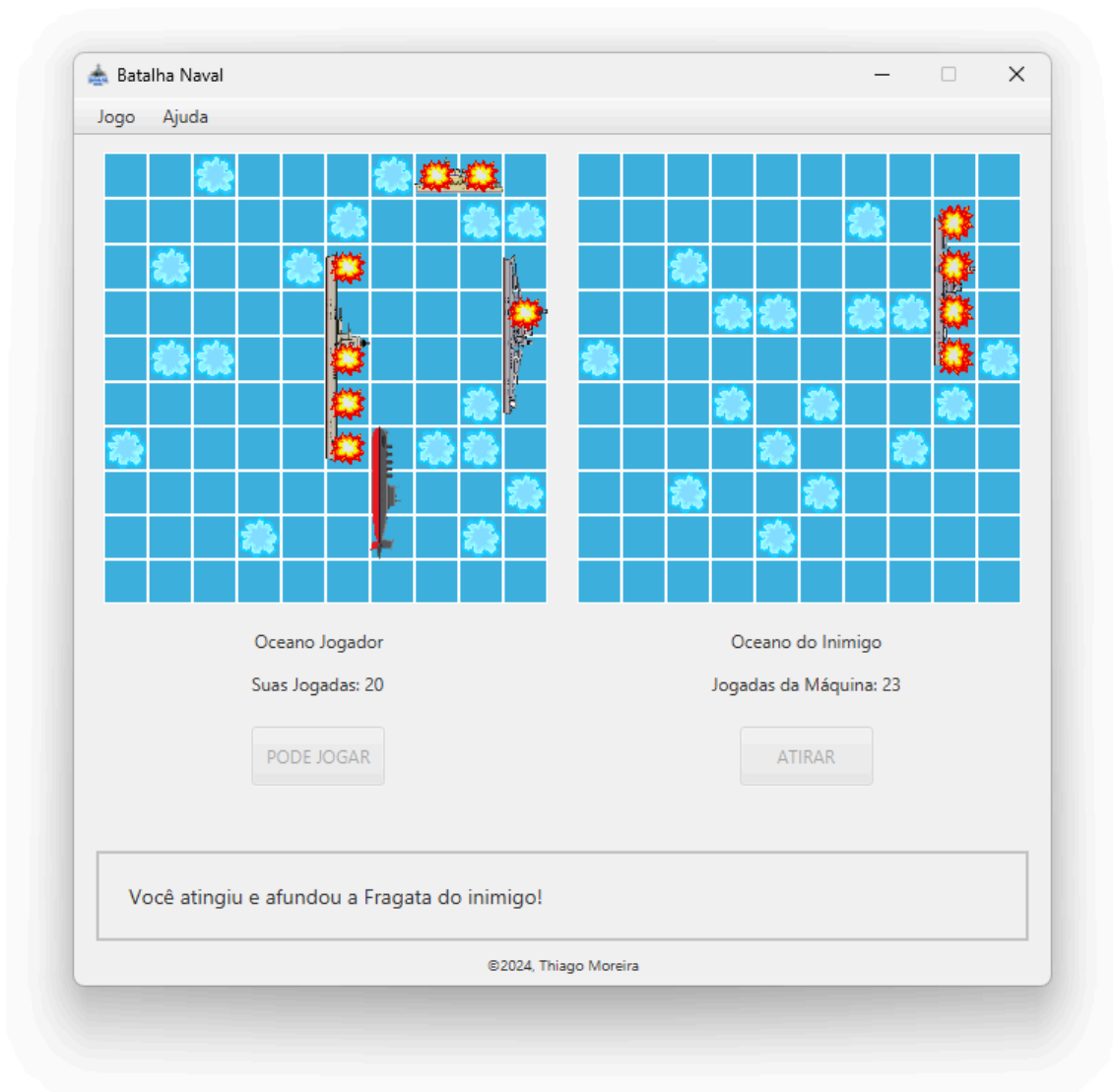
Ao clicar no botão “PODE JOGAR”, o tabuleiro do jogador é desativado, para impedir que o usuário reposicione seus navios durante a partida. Por outro lado, o tabuleiro do inimigo é ativado, para possibilitar que o jogador realize ataques no oceano inimigo. Quando o jogador seleciona uma célula válida do tabuleiro inimigo, um alvo é marcado sobre a célula, conforme a imagem acima, indicando para o jogador qual célula foi selecionada. Além disso, o botão “ATIRAR” é habilitado, para que o jogador confirme o disparo e verifique o resultado da ação.



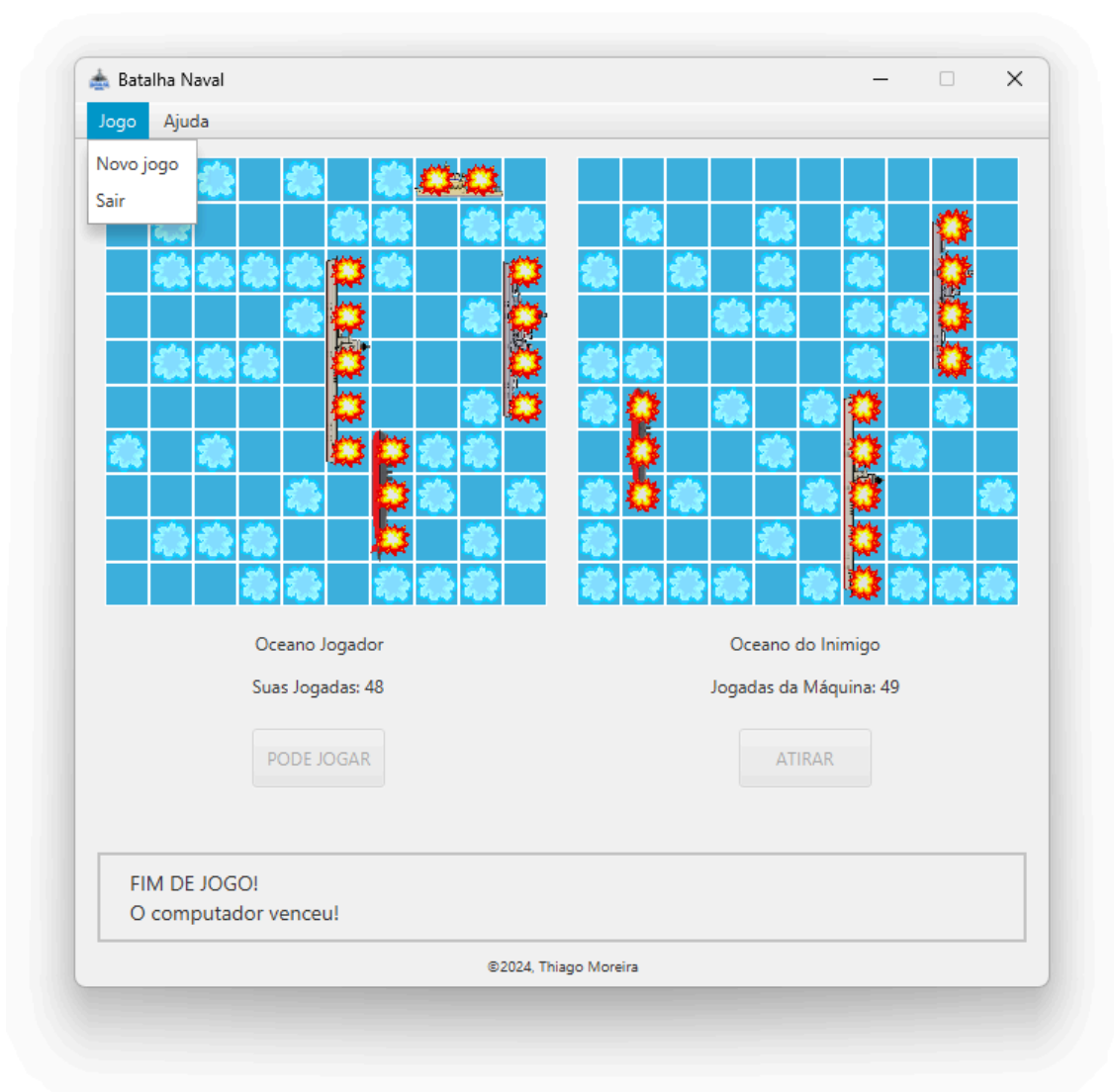
A imagem acima representa um tiro na água no oceano do inimigo que encerrou o turno do jogador e iniciou o turno da máquina. Esta, por sua vez, realizou dois disparos, atingindo uma parte do Destroyer do jogador e um tiro na água. O turno da máquina foi encerrado e se iniciou o turno do jogador. Note, no entanto, que o botão “ATIRAR” permanece desabilitado, uma vez que o jogador ainda não selecionou uma célula válida para realizar um disparo.



A imagem acima exibe a representação de disparos do jogador em um navio inimigo. O navio permanece oculto, enquanto não for completamente destruído.

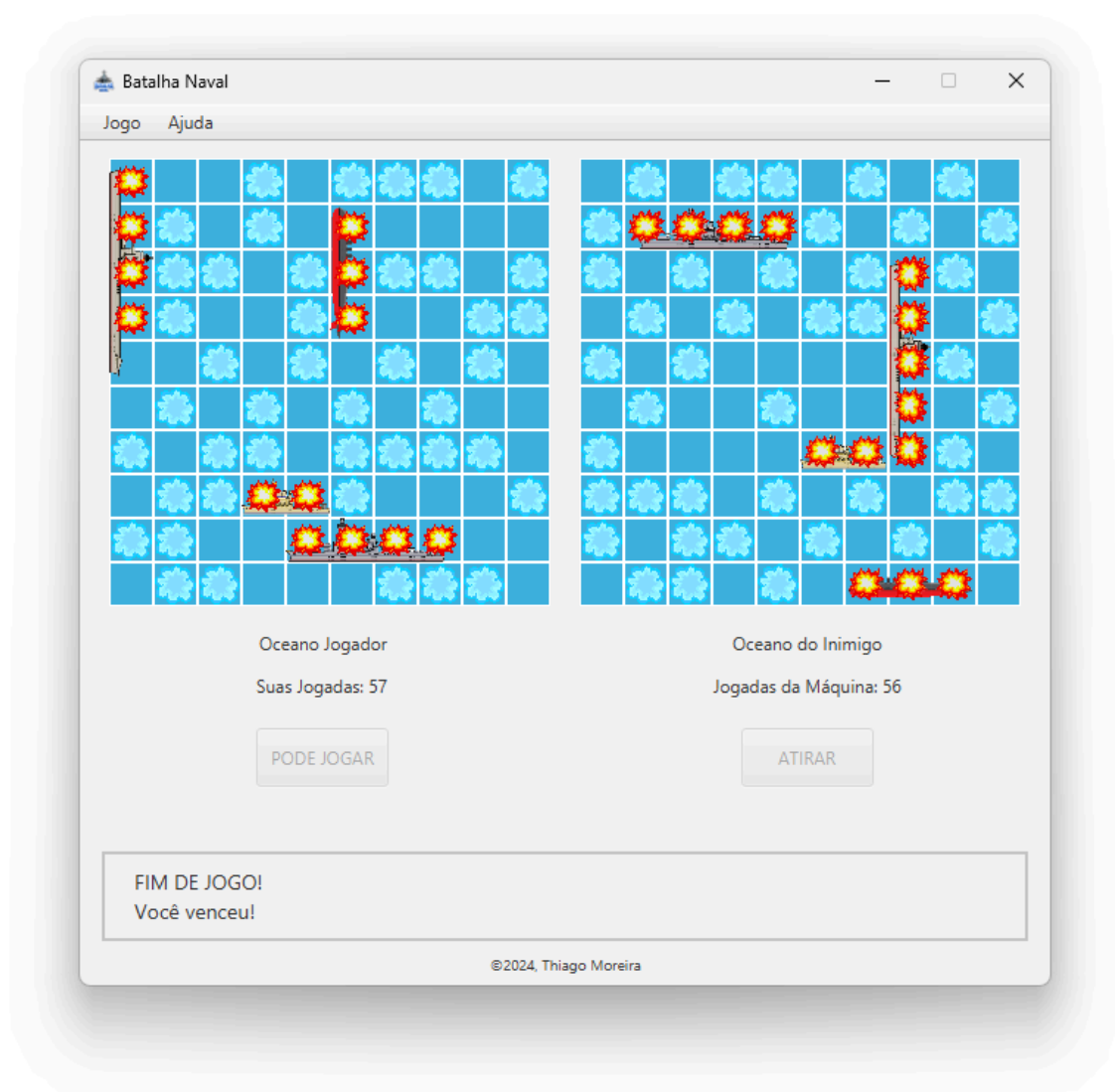


A partir do momento em que o navio inimigo é completamente destruído, sua imagem é exibida no tabuleiro e revela o tipo do navio destruído. Ao mesmo tempo, uma mensagem no painel fornece a informação de qual navio inimigo o jogador afundou (no exemplo acima, a Fragata).



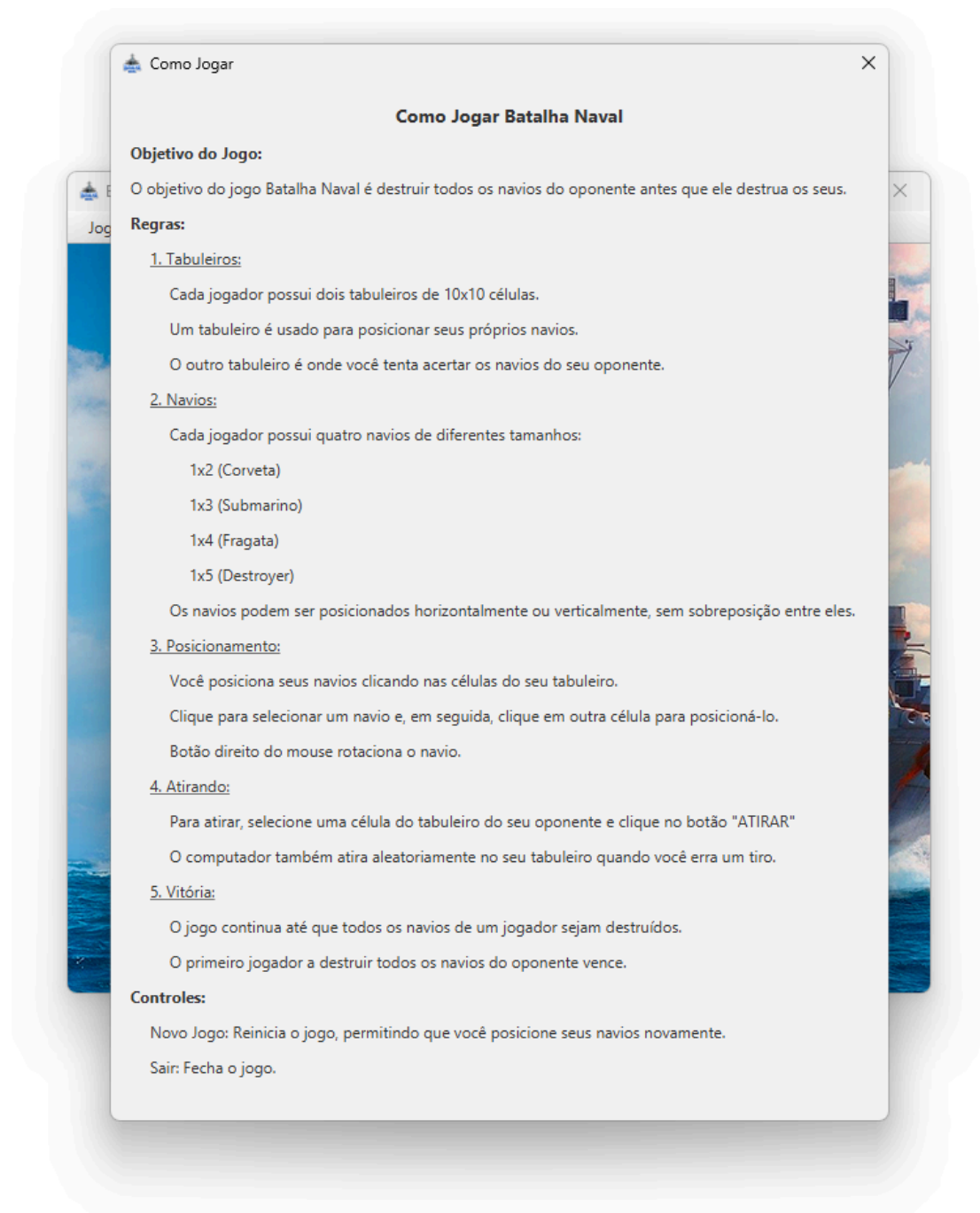
A imagem acima é uma representação da tela do jogo indicando a vitória do computador. Quando a partida se encerra, os tabuleiros e os botões são desativados e uma mensagem é exibida no painel, indicando o fim do jogo e o vencedor. Nesse cenário, as opções disponíveis para o jogador são:

- Iniciar um novo jogo, ao clicar na opção **Jogo** e depois em **Novo Jogo**;
- Sair do jogo, ao clicar
 - no ícone de fechar, na janela do jogo; ou
 - ao clicar em **Jogo** e depois em **Sair**;
- Abrir a janela **Como Jogar**, ao clicar em **Ajuda** e depois em **Como Jogar**;
- Abrir a janela **Sobre**, ao clicar em **Ajuda** e depois em **Sobre**.



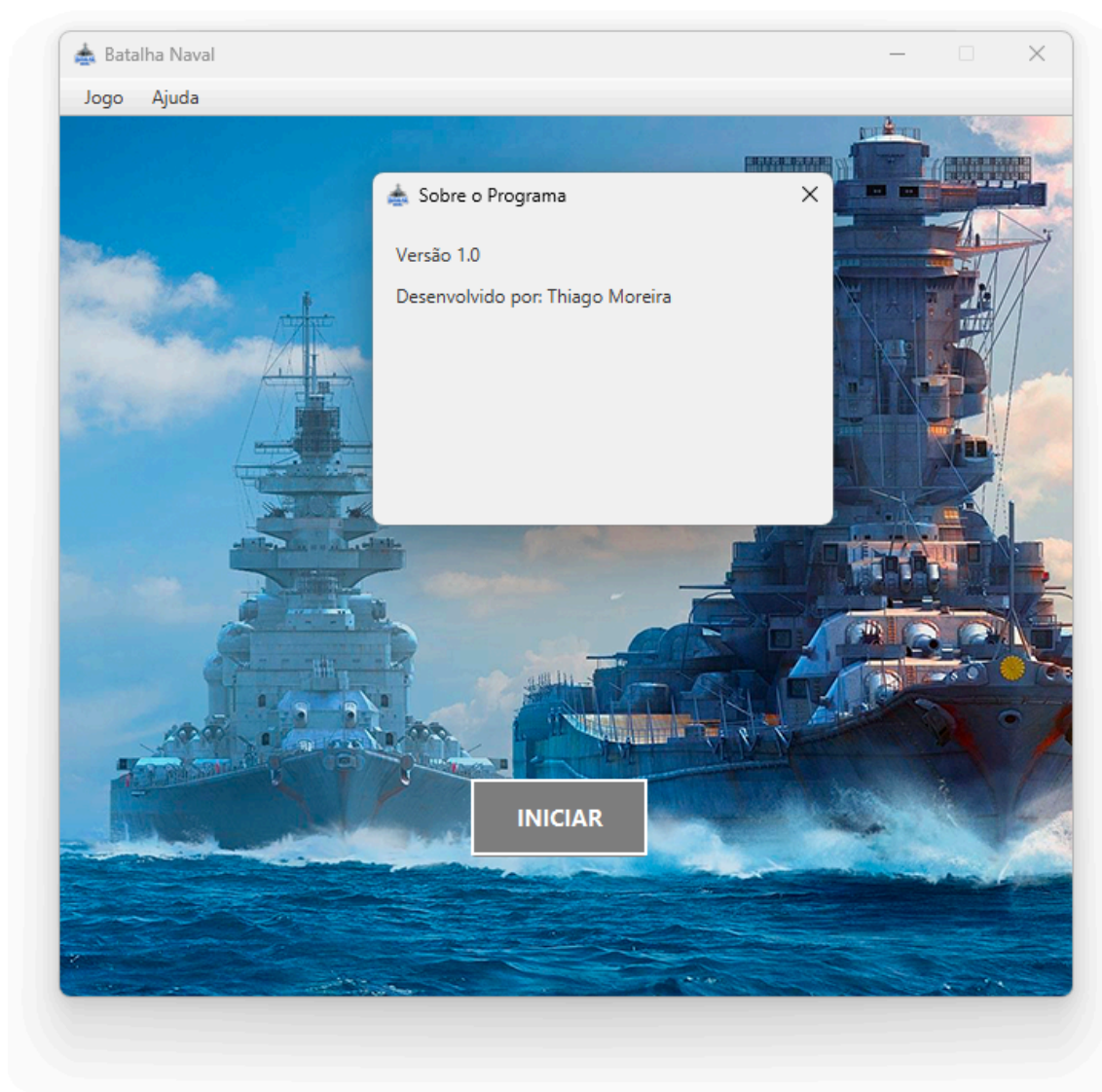
A imagem acima é uma representação da tela do jogo indicando a vitória do computador. Nesta imagem, é possível notar a exibição de todos os navios inimigos destruídos.

2.4.3. Tela Como Jogar



A função da tela **Como Jogar** é fornecer aos jogadores uma orientação clara e concisa sobre as regras e mecânicas do jogo, garantindo que todos, independentemente de sua experiência anterior, possam compreender e aproveitar a jogabilidade. Esta tela apresenta uma descrição passo a passo de como posicionar e rotacionar navios, realizar ataques e interpretar os resultados desses ataques. Além disso, inclui dicas sobre algumas aplicabilidades do programa, como as funções **Novo Jogo**, para reiniciar o jogo/programa, e **Sair**, para fechar o jogo/programa. Ao acessar a tela **Como Jogar**, os jogadores obtêm uma compreensão rápida e eficaz das operações básicas e avançadas, promovendo uma experiência de jogo mais satisfatória e acessível.

2.4.4. Tela Sobre



A função da tela **Sobre** é apresentar aos jogadores informações detalhadas sobre o projeto, incluindo a versão do programa e a equipe de desenvolvimento. Ao acessar esta tela, os jogadores têm a oportunidade de conhecer melhor o trabalho e o esforço investido no desenvolvimento do jogo, criando uma conexão mais pessoal e apreciativa com o projeto.

3. Estruturas e lógicas utilizadas

3.1. Definindo o posicionamento dos Navios

3.1.1. Corveta

```
public void definirPosicionamento() {  
    Random random = new Random();  
  
    this.orientacao = random.nextInt(2);  
    if (this.orientacao == 0) {  
        this.linha = random.nextInt(10);  
        this.coluna = random.nextInt(9);  
    } else {  
        this.linha = random.nextInt(9);  
        this.coluna = random.nextInt(10);  
    }  
}
```

Ao chamarmos o método **definirPosicionamento()**, da classe Corveta, um inteiro com valor 0 ou 1 será gerado aleatoriamente, para representar a orientação do navio, da seguinte forma:

- Inteiro igual a 0: Corveta será posicionada na horizontal;
- Inteiro igual a 1: Corveta será posicionada na vertical.

Posteriormente, serão definidas a linha e a coluna iniciais em que a Corveta estará posicionada, respeitando-se os limites do tabuleiro, de acordo com o tamanho do navio. Considerando que o tabuleiro possui 10 linhas (índices de 0 a 9) e 10 colunas (índices de 0 a 9) e que a Corveta possui tamanho 1x2, ou seja, ocupa **uma linha e duas colunas** ou **uma coluna e duas linhas** do tabuleiro, temos que:

- Se a Corveta estiver na horizontal, poderá ocupar qualquer linha inicial (de 0 a 9), mas sua coluna inicial deverá ser, no máximo, 8;
- Se a Corveta estiver na vertical, poderá ocupar qualquer coluna inicial (de 0 a 9), mas sua linha inicial deverá ser, no máximo, 8;

3.1.2. Submarino

```

public void definirPosicionamento() {
    Random random = new Random();

    this.orientacao = random.nextInt(2);
    if (this.orientacao == 0) {
        this.linha = random.nextInt(10);
        this.coluna = random.nextInt(8);
    } else {
        this.linha = random.nextInt(8);
        this.coluna = random.nextInt(10);
    }
}
}

```

Ao chamarmos o método **definirPosicionamento()**, da classe Submarino, um inteiro com valor 0 ou 1 será gerado aleatoriamente, para representar a orientação do navio, da seguinte forma:

- Inteiro igual a 0: Submarino será posicionado na horizontal;
- Inteiro igual a 1: Submarino será posicionado na vertical.

Posteriormente, serão definidas a linha e a coluna iniciais em que o Submarino estará posicionado, respeitando-se os limites do tabuleiro, de acordo com o tamanho do navio. Considerando que o tabuleiro possui 10 linhas (índices de 0 a 9) e 10 colunas (índices de 0 a 9) e que o Submarino possui tamanho 1x3, ou seja, ocupa **uma linha e três colunas** ou **uma coluna e três linhas** do tabuleiro, temos que:

- Se o Submarino estiver na horizontal, poderá ocupar qualquer linha inicial (de 0 a 9), mas sua coluna inicial deverá ser, no máximo, 7;
- Se o Submarino estiver na vertical, poderá ocupar qualquer coluna inicial (de 0 a 9), mas sua linha inicial deverá ser, no máximo, 7;

3.1.3. Fragata

```

public void definirPosicionamento() {
    Random random = new Random();

    this.orientacao = random.nextInt(2);
    if (this.orientacao == 0) {
        this.linha = random.nextInt(10);
        this.coluna = random.nextInt(7);
    } else {
        this.linha = random.nextInt(7);
        this.coluna = random.nextInt(10);
    }
}
}

```

Ao chamarmos o método **definirPosicionamento()**, da classe Fragata, um inteiro com valor 0 ou 1 será gerado aleatoriamente, para representar a orientação do navio, da seguinte forma:

- Inteiro igual a 0: Fragata será posicionada na horizontal;
- Inteiro igual a 1: Fragata será posicionada na vertical.

Posteriormente, serão definidas a linha e a coluna iniciais em que a Fragata estará posicionada, respeitando-se os limites do tabuleiro, de acordo com o tamanho do navio. Considerando que o tabuleiro possui 10 linhas (índices de 0 a 9) e 10 colunas (índices de 0 a 9) e que a Fragata possui tamanho 1x4, ou seja, ocupa **uma linha e quatro colunas** ou **uma coluna e quatro linhas** do tabuleiro, temos que:

- Se a Fragata estiver na horizontal, poderá ocupar qualquer linha inicial (de 0 a 9), mas sua coluna inicial deverá ser, no máximo, 6;
- Se a Fragata estiver na vertical, poderá ocupar qualquer coluna inicial (de 0 a 9), mas sua linha inicial deverá ser, no máximo, 6;

3.1.4. Destroyer

```
public void definirPosicionamento() {
    Random random = new Random();

    this.orientacao = random.nextInt(2);
    if (this.orientacao == 0) {
        this.linha = random.nextInt(10);
        this.coluna = random.nextInt(6);
    } else {
        this.linha = random.nextInt(6);
        this.coluna = random.nextInt(10);
    }
}
```

Ao chamarmos o método **definirPosicionamento()**, da classe Destroyer, um inteiro com valor 0 ou 1 será gerado aleatoriamente, para representar a orientação do navio, da seguinte forma:

- Inteiro igual a 0: Destroyer será posicionado na horizontal;
- Inteiro igual a 1: Destroyer será posicionado na vertical.

Posteriormente, serão definidas a linha e a coluna iniciais em que o Destroyer estará posicionado, respeitando-se os limites do tabuleiro, de acordo com o tamanho do navio. Considerando que o tabuleiro possui 10 linhas (índices de 0 a 9) e 10 colunas (índices de 0 a 9) e que o Destroyer possui tamanho 1x5, ou seja, ocupa **uma linha e cinco colunas** ou **uma coluna e cinco linhas** do tabuleiro, temos que:

- Se o Destroyer estiver na horizontal, poderá ocupar qualquer linha inicial (de 0 a 9), mas sua coluna inicial deverá ser, no máximo, 5;
- Se o Destroyer estiver na vertical, poderá ocupar qualquer coluna inicial (de 0 a 9), mas sua linha inicial deverá ser, no máximo, 5;

3.2. Reduzindo o tamanho de um Navio

```

    public void reduzirTamanho() {
        this.tamanho--;
        if (tamanho == 0) {
            this.ativo = false;
        }
    }
}

```

A cada vez que o método **reduzirTamanho()** de um navio qualquer é chamado, o atributo do navio chamado **tamanho** (int) é reduzido 1 unidade e uma verificação para detectar se o atributo tamanho atingiu o valor igual a 0 é realizada. Caso o atributo **tamanho** atinja um valor igual a 0, o atributo do navio chamado **ativo** (boolean) recebe o valor falso. Isso indica que aquele navio foi afundado, ou seja, todas as suas posições no tabuleiro foram atingidas por um disparo do jogador ou da máquina.

3.3. Matriz para o Tabuleiro de Jogo

```

package br.ufrn.imd.modelo;

* Esta classe representa o tabuleiro do jogo Batalha Naval.

public class Tabuleiro {
    public static final int DIM = 10; // dimensao do mapa
    private int mapa[][]; // Array bidimensional

    public Tabuleiro() {
        mapa = new int[DIM][DIM];
    }
}

```

A classe Tabuleiro possui dois atributos:

- Uma constante **DIM**, com valor igual a 10;
- Um array bidimensional de inteiros **mapa**, que representa o tabuleiro do jogo e tem dimensão igual a [DIM] x [DIM].

3.3.1. Método para iniciar o Tabuleiro

```

    public void IniciarTabuleiro() {
        for (int i = 0; i < DIM; i++) {
            for (int j = 0; j < DIM; j++) {
                mapa[i][j] = 0;
            }
        }
    }
}

```

Inicialmente, todas as posições do mapa serão preenchidas com o valor 0, para que, posteriormente, os navios sejam posicionados no tabuleiro.

3.3.2. Método para posicionar os navios no Tabuleiro

```
public void posicionarNavio(Navio navio) {
    if (navio.getOrientacao() == 0) { // horizontal
        for (int i = navio.getColuna(); i < navio.getColuna() + navio.getTamanho(); i++) {
            mapa[navio.getLinha()][i] = navio.getId();
        }
    } else { // vertical
        for (int i = navio.getLinha(); i < navio.getLinha() + navio.getTamanho(); i++) {
            mapa[i][navio.getColuna()] = navio.getId();
        }
    }
}
```

O método **posicionarNavio(Navio navio)** é responsável por posicionar um navio no tabuleiro de jogo (representado por uma matriz mapa). Esse método leva um objeto Navio como parâmetro, que contém informações sobre a posição inicial, tamanho, orientação (horizontal ou vertical) e um identificador único do navio. Abaixo, uma explicação detalhada do que o código faz:

- Verificação da orientação do Navio:
 - Inicialmente, o método verifica a orientação do navio usando **navio.getOrientacao()**.
 - Se a orientação for 0, o navio será colocado horizontalmente.
 - Se a orientação for 1, o navio será colocado verticalmente.
- Posicionamento Horizontal:
 - Para um navio horizontal, o código itera sobre as colunas, começando da coluna inicial (**navio.getColuna()**) até a posição final, que é determinada pelo tamanho do navio (**navio.getTamanho()**).
 - Em cada iteração, o método atribui o identificador do navio (**navio.getId()**) às células apropriadas na linha especificada (**navio.getLinha()**) do tabuleiro.
- Posicionamento Vertical:
 - Para um navio vertical, o código itera sobre as linhas, começando da linha inicial (**navio.getLinha()**) até a posição final determinada pelo tamanho do navio (**navio.getTamanho()**).
 - Em cada iteração, o método atribui o identificador do navio (**navio.getId()**) às células apropriadas na coluna especificada (**navio.getColuna()**) do tabuleiro.

Em resumo, o método **posicionarNavio(Navio navio)** coloca um navio no tabuleiro de jogo, marcando as células da matriz mapa de acordo com a orientação, posição inicial e tamanho do navio.

3.3.3. Método para limpar uma posição antiga do navio

```

public void limparPosicaoAntigaDoNavio(Navio navio) {
    if (navio.getOrientacao() == 0) { // horizontal
        for (int i = navio.getColuna(); i < navio.getColuna() + navio.getTamanho(); i++) {
            mapa[navio.getLinha()][i] = 0;
        }
    } else { // vertical
        for (int i = navio.getLinha(); i < navio.getLinha() + navio.getTamanho(); i++) {
            mapa[i][navio.getColuna()] = 0;
        }
    }
}
}

```

Nos casos em que um navio é reposicionado ou rotacionado, faz-se necessário limpar o posicionamento antigo do navio no tabuleiro. O método acima percorre as posições antigas do navio e substitui os valores que representam o ID do navio pelo valor 0, que representa a água no tabuleiro.

3.3.4. Métodos para verificar se há navio(s) em uma determinada localização

```

public boolean verificarSeHaNavioNaLocalizacao(Navio navio) {
    if (navio.orientacao == 0) {
        for (int i = navio.coluna; i < navio.coluna + navio.tamanho; i++) {
            if (mapa[navio.linha][i] != 0) {
                return false;
            }
        }
    } else {
        for (int i = navio.linha; i < navio.linha + navio.tamanho; i++) {
            if (mapa[i][navio.coluna] != 0) {
                return false;
            }
        }
    }

    return true;
}

public boolean verificarSeHaNavioNaLocalizacao(int linha, int coluna, Navio navio) {
    if (navio.orientacao == 0) {
        for (int i = coluna; i < coluna+navio.tamanho; i++) {
            if (mapa[linha][i] != 0 && mapa[linha][i] != navio.id) {
                return false;
            }
        }
    } else {
        for (int i = linha; i < linha+navio.tamanho; i++) {
            if (mapa[i][coluna] != 0 && mapa[i][coluna] != navio.id) {
                return false;
            }
        }
    }

    return true;
}

```

```

public boolean verificarSeHaNavioNaLocalizacao(Navio navio, int orientacao) {
    if(orientacao == 0) {
        for (int i = navio.coluna; i < navio.coluna+navio.tamanho; i++) {
            if(mapa[navio.linha][i] != 0 && mapa[navio.linha][i] != navio.id) {
                return false;
            }
        }
    } else {
        for (int i = navio.linha; i < navio.linha+navio.tamanho; i++) {
            if(mapa[i][navio.coluna] != 0 && mapa[i][navio.coluna] != navio.id) {
                return false;
            }
        }
    }

    return true;
}

```

Os métodos acima foram construídos utilizando polimorfismo de sobrecarga e são responsáveis por verificar se algum navio (ou parte dele) está posicionado em uma determinada localização do tabuleiro.

O primeiro método recebe como parâmetro apenas um **Navio** e é utilizado no processo de posicionamento aleatório dos navios no tabuleiro, para impedir que um navio seja posicionado sobre outro(s).

O segundo método recebe os inteiros **linha** e **coluna** e um objeto Navio **navio** como parâmetros. Este método é utilizado no processo de reposicionamento dos navios, para verificar se a nova localização em que se pretende reposicionar o navio já está ocupada por outro(s) navio(s) ou parte dele(s). Vale ressaltar que a linha e a coluna que serão passadas como parâmetro neste método representam a nova posição em que se pretende posicionar o navio, que será obtida a partir do clique do usuário na célula escolhida.

O terceiro método recebe um objeto Navio **navio** e um inteiro **orientacao** como parâmetros. Este método é utilizado no processo de rotacionamento dos navios, para verificar se a nova localização em que se pretende rotacionar o navio já está ocupada por outro(s) navio(s) ou parte dele(s).

3.3.5. Método para verificar se um navio foi atingido

```

public boolean verificarSeAcertouNavio(int linha, int coluna) {
    if (mapa[linha][coluna] != 0) {
        return true;
    }
    return false;
}

```

O método **verificarSeAcertouNavio(int linha, int coluna)** recebe como parâmetros uma linha e uma coluna e verifica se o valor inteiro alocado nessa posição da matriz mapa é diferente de 0 (água). Caso essa condição seja verdadeira, será retornado o valor **true** pelo método, indicando que um navio foi atingido. Caso contrário, o método

retornará o valor **false**, indicando que o tiro atingiu a água.

3.3.6. Método para identificar o navio atingido

```
public int retornaValor(int i, int j) {  
    return this.mapa[i][j];  
}
```

Para que a lógica do jogo se torne funcional, é fundamental identificar o resultado do disparo realizado pelo jogador ou pela máquina. Para isso, o método **retornaValor(int i, int j)** é responsável por retornar o valor do inteiro que está na linha *i* e na coluna *j* da matriz **mapa**. Os resultados possíveis são:

- 0 indica que o disparo foi na água;
- 1 indica que o disparo atingiu uma Corveta;
- 2 indica que o disparo atingiu um Submarino;
- 3 indica que o disparo atingiu uma Fragata; e
- 4 indica que o disparo atingiu um Destroyer.

3.4. Repositório para Jogadas

```
package br.ufrn.imd.dao;  
  
/**  
 * Esta classe e um repositorio que registra todas as jogadas realizadas por um jogador na partida.  
 * <br>  
 * A principio, esta classe possui apenas um atributo (jogadas).  
 */  
  
import java.util.ArrayList;  
  
public class JogadasDAO {  
    private ArrayList<int[]> jogadas; //ArrayList que armazena todas as jogadas realizadas por um jogador.  
  
    /**  
     * Metodo construtor que cria um novo repositorio de jogadas.  
     */  
    public JogadasDAO() {  
        jogadas = new ArrayList<int[]>();  
    }  
}
```

A classe de repositório de jogadas, denominada **JogadasDAO**, foi desenvolvida para registrar todas as jogadas realizadas pelo jogador/computador durante a partida de Batalha Naval. Esta classe atua como uma interface entre a lógica do jogo e o armazenamento persistente dos dados das jogadas, garantindo que cada ataque realizado pelo jogador seja devidamente registrado com informações precisas. Ao centralizar o gerenciamento das jogadas, a classe assegura a eficiência das ações realizadas pelo usuário, permitindo que o jogo mantenha um histórico preciso e eficiente das ações dos jogadores e impedindo que jogadas repetidas sejam realizadas pelo jogador/computador.

3.4.1. Método para adicionar jogadas ao repositório

```

public void addJogadas(int linha, int coluna) {
    int[] dados = new int[2];
    dados[0] = linha;
    dados[1] = coluna;
    jogadas.add(dados);
    System.out.println("Jogada inserida com sucesso!");
}

```

O método acima é responsável por adicionar ao ArrayList **jogadas** um vetor com dois inteiros, que representa a jogada realizada pelo jogador/computador, no formato {linha, coluna}.

3.4.2. Método para verificar se uma jogada já foi realizada

```

public boolean verificarSeJaHouveEssaJogada(int linha, int coluna) {
    for(int[] jogada : jogadas) {
        if (linha == jogada[0] && coluna == jogada[1]) {
            return true;
        }
    }
    return false;
}

```

O método **verificarSeJaHouveEssaJogada(int linha, int coluna)** recebe como parâmetros uma linha e uma coluna, que representam a posição do tabuleiro em que o jogador/computador pretende realizar um disparo. Nesse sentido, a estrutura de repetição **for** irá percorrer todas as jogadas já registradas no ArrayList **jogadas** e verificar se a jogada pretendida pelo jogador/computador já foi realizada. Caso essa condição seja verdadeira, o método retorna o valor boolean **true**. Caso contrário, o valor boolean **false** será retornado.

3.4.3. Método para calcular a quantidade de jogadas já feitas

```

public int retornaQuantidadeDeJogadas() {
    return jogadas.size();
}

```

O método **retornaQuantidadeDeJogadas()** retorna um inteiro que representa a quantidade de jogadas já realizadas pelo jogador/computador. Este valor é obtido a partir do método **size()**, cuja funcionalidade é retornar um valor inteiro que representa o número de elementos em uma lista.

4. Conclusão

O desenvolvimento do jogo Batalha Naval em Java utilizando conceitos de POO e JavaFX foi um exercício valioso na aplicação de princípios de design de software e padrões de projeto. A separação clara de responsabilidades e a aplicação de padrões

como MVC contribuíram para a criação de um código bem estruturado e de fácil manutenção. A utilização do JavaFX permitiu a criação de uma interface gráfica interativa e amigável. Este projeto não só serviu como uma excelente prática de programação orientada a objetos, mas também demonstrou a importância de um bom design de software na criação de aplicações complexas.

5. Referências

JavaFX Documentation: <https://openjfx.io/>

Oracle Java Documentation: <https://docs.oracle.com/en/java/>